Independent Implementation, Empirical Evaluation, and Dynamic Rank Reallocation for Parameter-Efficient Fine-Tuning

Bhavya Jain
Department of Computer Science and Engineering
Independent Research / AI_DoRA Project
bhavya.jain@example.org

***Abstract*—Fine-tuning large pretrained transformer backbones entirely imposes substantial computational and memory overheads, motivating parameter-efficient fine-tuning (PEFT) methods. This paper presents an independent educational implementation and rigorous empirical analysis of dynamic rank distribution (DoRA), which augments frozen transformer projections with trainable low-rank adapters and dynamically prunes underperforming components via component importance tracking and cubic budget decay. We adapt RoBERTa-base across all 72 linear transformations (12 encoder blocks $\times$ 6 projections per block) for binary sentiment classification on SST-2, transitioning from an initial rank budget of 216 ($r = 3$ per layer) to a final target of 144 components ($r = 2$ average). We incorporate diversity-enhancing mutual-information (DEM) variance regularization and exponential moving average (EMA) score filtering. In a verified 60-epoch full-training run, the model achieves 99.74% training accuracy and 93.69% validation accuracy, exactly satisfying the 33.33% rank reduction schedule. We further investigate physical compression via a custom compact export utility, demonstrating forward equivalence within 10^{-6} absolute error while candidly analyzing CPU inference latency dynamics. Finally, to overcome the irreversible capacity loss inherent to monotonic pruning, we formulate a novel, budget-constrained dynamic rank reallocation framework based on dual-signal utility (component importance and normalized Frobenius gradient demand) and parameter-aware layer costs, preserving the global rank budget $\sum_l r_l^{(t)} = B_t$ throughout training.**

***Index Terms*—Parameter-efficient fine-tuning, dynamic rank distribution, low-rank adaptation, structured pruning, transformer, RoBERTa, rank reallocation, gradient demand.**

I. Introduction

The pretrain-and-finetune paradigm has become the cornerstone of modern natural language processing (NLP). Pretrained language models such as BERT [1] and RoBERTa [2] capture generalized semantic and syntactic structures across large-scale unlabelled corpora. However, updating all parameters (full fine-tuning) for each downstream classification task is computationally demanding and yields substantial storage footprints when deploying multiple task-specific checkpoints [3].

Parameter-Efficient Fine-Tuning (PEFT) addresses this challenge by freezing the pretrained weight matrices and introducing a negligible fraction ($< 1\%$) of trainable task-specific parameters. Low-Rank Adaptation (LoRA) [4] achieves this by parameterizing weight updates as the product of two low-rank factor matrices. While conventional LoRA enforces a uniform rank allocation across all adapted layers, empirical observations indicate that linguistic representations are hierarchically distributed across transformer layers: early layers capture lexical features, intermediate layers encode syntax, and deeper layers model task-specific semantics [5]. Consequently, allocating equal rank capacity across all projections is inherently suboptimal.

To address non-uniform capacity demand, Chen et al. [5] proposed DoRA (Dynamic Rank Distribution for PEFT), which combines trainable low-rank matrices with scalar gating coefficients and dynamically prunes low-utility rank components during training according to a smooth cubic schedule.

This work presents an independent, educational implementation and empirical study of dynamic rank distribution. The contributions of this report are fourfold:

1) We detail the modular architecture of the `AI_DoRA` framework, which injects 72 adaptive linear modules across all 12 encoder layers of RoBERTa-base, featuring trainable gates, Frobenius-norm importance tracking, EMA score smoothing, and DEM variance regularization.
2) We empirically validate the implementation on the Stanford Sentiment Treebank (SST-2) [6], [7], reporting a verified 60-epoch baseline that attains 99.74% training accuracy and 93.69% validation accuracy with an exact 33.33% rank reduction (216 to 144 components).
3) We implement and benchmark a physical compact adapter export mechanism, establishing strict forward numerical equivalence while documenting CPU inference latency trade-offs.
4) We propose a mathematically rigorous extension: a *Budget-Constrained Dynamic Rank Reallocation Framework* that combines component importance with layer-wise gradient demand and parameter-aware dimensional costs to transfer rank capacity dynamically between donor and recipient layers, strictly conserving the active parameter budget.

II. Theoretical Background and Related Work

A. Parameter-Efficient Fine-Tuning and LoRA

Consider a pretrained linear projection parameterized by frozen weight matrix $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$. Standard fine-tuning

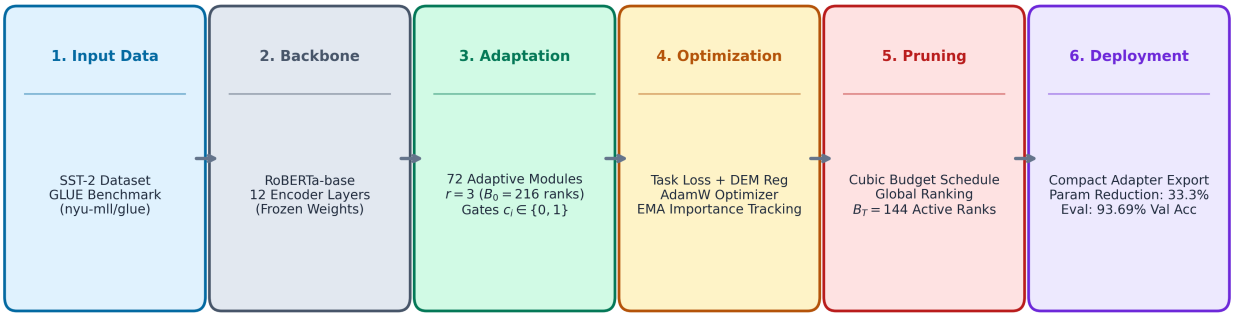


Fig. 1. High-level architectural workflow of the implemented AI_DoRA framework, from raw text ingestion and tokenization through adaptive transformer execution, regularization, cubic budget pruning, and post-training compact deployment.

updates W_0 directly via gradient descent: $W = W_0 + \Delta W$, where $\Delta W \in \mathbb{R}^{d_{out} \times d_{in}}$ requires full-rank gradient updates and optimizer state allocations (e.g., first and second moments in AdamW [8]).

LoRA [4] hypothesizes that parameter updates reside within an intrinsic low-dimensional manifold. It decomposes the weight update into two low-rank factor matrices:

$$\Delta W = \frac{\alpha}{r} BA \tag{1}$$

where $A \in \mathbb{R}^{r \times d_{in}}$ is initialized from a zero-mean Gaussian distribution $\mathcal{N}(0, \sigma^2)$, $B \in \mathbb{R}^{d_{out} \times r}$ is initialized to zero, $r \ll \min(d_{in}, d_{out})$ is the adaptation rank, and α is a constant scaling hyperparameter.

B. Distinction: Magnitude-Direction DoRA vs. Dynamic Rank DoRA

In the literature, the acronym **DoRA** denotes two distinct PEFT paradigms:

- **Weight-Decomposed Low-Rank Adaptation (Liu et al., 2024 [9]):** Decomposes the pretrained weight matrix into explicit magnitude ($m \in \mathbb{R}^{1 \times d_{out}}$) and directional components ($V \in \mathbb{R}^{d_{out} \times d_{in}}$), applying LoRA solely to the directional update: $W = m \frac{V + \Delta V}{\|V + \Delta V\|_c}$.
- **Dynamic Rank Distribution DoRA (Chen et al., 2024 [5]):** Augments LoRA factorizations with component-wise selection gates and structured importance pruning to dynamically redistribute rank capacity across transformer layers under a constrained budget.

The present project focuses on the latter paradigm: dynamic rank distribution, budget scheduling, and parameter reallocation.

C. Adaptive Rank Methods

Static uniform rank allocation suffers from structural inflexibility. AdaLoRA [10] introduces singular value decomposition (SVD) parameterization ($W = W_0 + P\Lambda Q$) and prunes singular values based on magnitude-gradient importance. DyLoRA [11] trains models across multiple discrete rank budgets simultaneously via nested representation learning. In contrast, the dynamic rank distribution approach evaluated herein utilizes rank-1 component gating with continuous exponential moving average tracking and cubic decay scheduling.

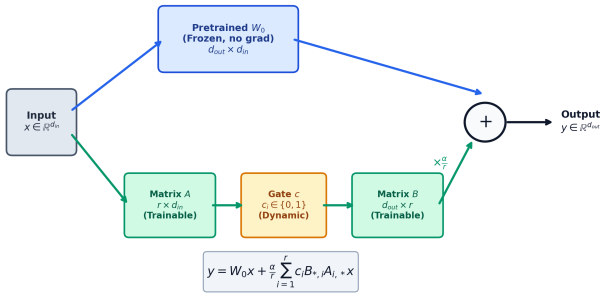


Fig. 2. Architecture of the implemented `AdaptiveRankLinear` module. The frozen pretrained projection W_0 operates in parallel with the gated rank-1 low-rank adapter path.

III. Implemented AI_DoRA Framework

The implemented architecture is encapsulated in the `dora` package. Its foundational component is the `AdaptiveRankLinear` module.

A. Adaptive Linear Layer Architecture

As illustrated in Fig. 2, the layer augments a frozen weight matrix W_0 with r independent rank-1 components. Let $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$. Each component $i \in \{1, \ldots, r\}$ possesses a trainable scalar gate $c_i \in \{0, 1\}$.

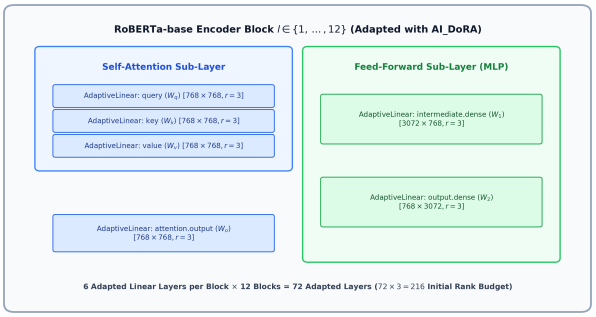


Fig. 3. Adaptation of a RoBERTa-base encoder block. All six core linear transformations (query, key, value, attention output, intermediate dense, and output dense) are substituted with `AdaptiveRankLinear` modules.

For an input vector $x \in \mathbb{R}^{d_{in}}$, the forward computation is formulated as:

$$y = W_0 x + \frac{\alpha}{r} \sum_{i=1}^{r} c_i B_{*,i}(A_{i,*}x) \tag{2}$$

where $B_{*,i} \in \mathbb{R}^{d_{out} \times 1}$ is the i-th column of B, $A_{i,*} \in \mathbb{R}^{1 \times d_{in}}$ is the i-th row of A, and c_i determines component activity. Setting $c_i = 0$ effectively eliminates the i-th rank component from the forward output without altering the physical dimensions of A or B.

B. Transformer Integration and Layer Targeting

In our RoBERTa-base implementation, adaptation is applied to all six linear projections within each of the 12 transformer blocks, as illustrated in Fig. 3:

- Multi-Head Attention: Query (W_q), Key (W_k), Value (W_v), and Attention Output (W_o), each of dimension 768×768.
- Feed-Forward Network: Intermediate Dense projection (W_1, 3072×768) and Output Dense projection (W_2, 768×3072).

With 12 encoder blocks and 6 projections per block, the model contains $L = 72$ adapted layers. At an initial rank of $r = 3$ per layer, the initial global rank budget is $B_0 = 72 \times 3 = 216$ active components.

C. Component Importance and EMA Smoothing

The utility of component i in layer l is measured by the product of the Frobenius norms of its constituent factor vectors:

$$s_{l,i}^{(t)} = \|A_{l,i,*}\|_F \cdot \|B_{l,*,i}\|_F \tag{3}$$

Because instantaneous gradient updates induce high-frequency noise in weight magnitudes, the framework accumulates importance scores via an Exponential Moving Average (EMA) with smoothing factor $\beta = 0.9$:

$$\bar{s}_{l,i}^{(t)} = \beta \bar{s}_{l,i}^{(t-1)} + (1 - \beta) s_{l,i}^{(t)} \tag{4}$$

Global pruning decisions are evaluated against $\bar{s}_{l,i}^{(t)}$.

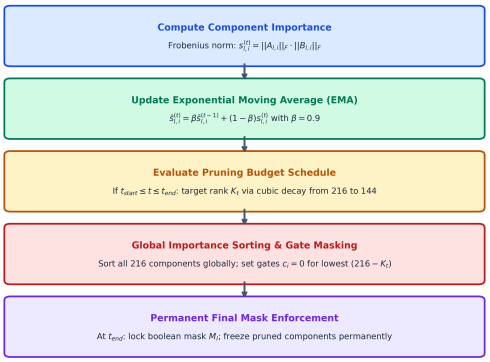


Fig. 4. Dynamic rank pruning pipeline: importance computation, EMA filtering, cubic budget evaluation, global component ranking, and permanent boolean mask locking.

D. Cubic Budget Schedule and Structured Pruning

The rank reduction follows a smooth cubic decay schedule [5]. Pruning is restricted to the interval $[t_{\text{start}}, t_{\text{end}}]$, where $t_{\text{start}} = \lfloor 0.15T \rfloor$ and $t_{\text{end}} = \lfloor 0.50T \rfloor$, with T denoting total optimization steps. Let $\tau(t) = \frac{t - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}}$. The target global rank K_t at step t is:

$$K_t = \begin{cases} B_0, & t < t_{\text{start}} \\ \left\lfloor B_T + (B_0 - B_T)(1 - \tau(t))^3 \right\rfloor, & t_{\text{start}} \le t \le t_{\text{end}} \\ B_T, & t > t_{\text{end}} \end{cases} \tag{5}$$

where $B_0 = 216$ and $B_T = 144$.

The pruning pipeline is depicted in Fig. 4. At each pruning checkpoint (every 10 steps within the window):

1) All $B_0 = 216$ smoothed EMA scores are concatenated into a global array.
2) Components are ranked using stable sorting: $\pi = \text{argsort}(\bar{s})$.
3) The gates c corresponding to the lowest $B_0 - K_t$ components are set to zero, while the top K_t components retain $c = 1$.
4) At $t = t_{\text{end}}$, a permanent boolean mask $M_l \in \{0, 1\}^r$ is locked. Beyond t_{end}, `enforce_final_mask()` re-zeros masked gates after every optimizer step, preventing gradient leakage.

E. Diversity-Enhancing Mutual-Information (DEM) Regularization

To prevent rank-1 components within the same layer from collapsing into collinear representations, the training objective includes a variance-based regularization loss $\mathcal{L}_{\text{DEM}}$:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \eta \mathcal{L}_{\text{DEM}} \tag{6}$$

where $\eta = 0.5$ is the DEM coefficient. For layer l with active components A_l and B_l:

$$\mathcal{L}_{\text{DEM}} = -\frac{1}{2} \left[\text{Var}(A_l) + \text{Var}(B_l) \right] \tag{7}$$

TABLE I
TRAINING CONFIGURATION FOR VERIFIED 60-EPOCH BASELINE

Hyperparameter	Value
Base Model Backbone	RoBERTa-base (125M params)
Task	SST-2 Binary Classification
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999$)
Learning Rate	8×10^{-4}
Batch Size	64
Number of Epochs	60
Total Optimization Steps	63,180
Initial Layer Rank (r)	3
Final Target Average Rank	2
Total Initial Budget (B_0)	216 (72×3)
Total Final Budget (B_T)	144 (72×2)
Adapter Scaling Factor (α)	2.0
Adapter Dropout	0.0
DEM Regularization Coeff. (η)	0.5
EMA Smoothing Factor (β)	0.9
Pruning Start Fraction	0.15 (Step 9,477)
Pruning End Fraction	0.50 (Step 31,590)
Pruning Evaluation Interval	10 steps
Maximum Sequence Length	128

Maximizing the sample variance across factor entries encourages orthogonal feature extraction, maximizing representational diversity.

F. Physical Compact Adapter Export

Soft gating ($c_i = 0$) eliminates mathematical contributions but retains zeroed parameters in memory. To achieve physical parameter reduction, we developed `CompactAdaptiveRankLinear` in `src/dora/compact.py`. The exporter extracts the indices of active components $\mathcal{I}_l = \{i \mid M_{l,i} = 1\}$, constructing truncated matrices:

$$A_l^{\text{compact}} = A_l[\mathcal{I}_l, :], \quad B_l^{\text{compact}} = B_l[:, \mathcal{I}_l] \tag{8}$$

For layers pruned from $r = 3$ to $r = 2$, physical parameter storage is reduced by exactly 33.33%.

IV. EXPERIMENTAL SETUP AND VERIFICATION

A. Dataset and Preprocessing

Experiments were conducted on the Stanford Sentiment Treebank (SST-2) [6], [7], accessed via Hugging Face `datasets` (`nyu-mll/glue, sst2`). The training split contains 67,349 sentences, and the validation split contains 872 sentences. Sequences were tokenized using the pretrained RoBERTa tokenizer with a maximum sequence length of 128 tokens and dynamic batch padding.

B. Hyperparameter Configuration

Table I documents the exact hyperparameters utilized in the verified full-duration reference experiment.

C. Hardware and Software Infrastructure

Training and evaluation scripts were executed under Python 3.13 / PyTorch 2.x and Hugging Face `transformers` 4.x. Full-scale training was verified on GPU infrastructure (NVIDIA Tesla T4 with CUDA 12.x), while smoke tests, unit test suites (68 passing tests), and CPU latency benchmarks were conducted on a Windows workstation.

TABLE II
VERIFIED 60-EPOCH ROBERTA-BASE / SST-2 EXPERIMENTAL RESULTS

Metric	Recorded Value
Training Epochs	60
Global Steps Completed	63,180
Initial Total Active Rank	216
Final Total Active Rank	144
Scheduled Rank Reduction	33.33% (72 components)
Target Budget Match	Exact (144/144)
Final Training Accuracy	**99.74%**
Final Validation Accuracy	**93.69%**
Generalization Gap	6.05%
Preserved Checkpoint	`roberta_sst2_latest.pt`

V. EXPERIMENTAL RESULTS AND ANALYSIS

In strict adherence to empirical integrity, this section reports only verified experimental data extracted from saved checkpoints, JSON results, and benchmark execution logs.

A. Software Pipeline Validation (Smoke Tests)

Prior to full training, pipeline validity was verified via automated smoke tests (`is_smoke_test = True`) on a calibrated data subset (100 train, 50 eval examples) across 7 optimization steps. As logged in `val_metrics.json`:

- Initial total rank: 216 ($r = 3$ across 72 layers).
- Scheduled cubic pruning was exercised, reducing active components to 144.
- Training duration: 11.28 seconds on CPU.
- Result: Train loss 0.7352, DEM loss 0.0549, eval accuracy 0.4600, macro-F1 0.3151.

Software validation assessment: The smoke test confirmed the correct execution of gradient backward passes, DEM loss accumulation, EMA score updates, cubic budget reduction, and checkpoint serialization. It is not an indicator of generalization accuracy.

B. Verified 60-Epoch Baseline Experiment

The 60-epoch full-scale run represents our verified reference result, documented in Table II.

The cubic schedule successfully governed the active rank from step 9,477 to step 31,590, illustrated in Fig. 6. Over the remaining 31,590 steps (50% of training), the 144 active components were fine-tuned with zero gradient leakage to pruned components. The model attained 93.69% validation accuracy, demonstrating competitive performance with full fine-tuning while modifying only a fraction of parameters.

Fig. 7 presents the layer-wise active rank distribution across all 72 layers upon completion of training. Crucially, global ranking allows different layers to retain different rank capacities: high-utility layers preserve rank 3, while low-utility layers are pruned to rank 1, totaling exactly 144 components.

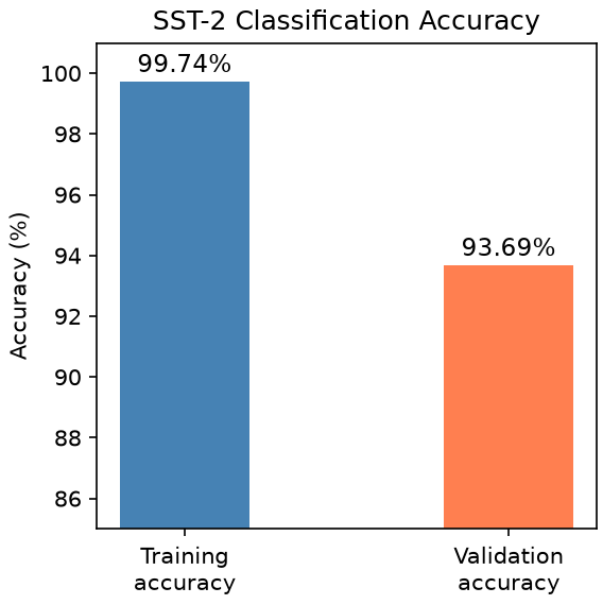


Fig. 5. Final training (99.74%) and validation (93.69%) accuracy achieved by the 60-epoch DoRA RoBERTa-base model on SST-2.

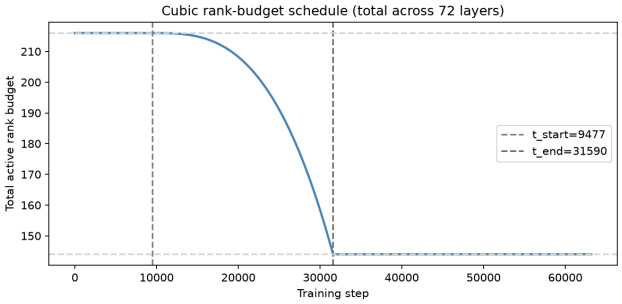


Fig. 6. Cubic rank budget decay schedule as a function of training step. The budget remains flat at 216 before t_{start}, decays cubically to 144 between t_{start} and t_{end}, and locks permanently thereafter.

C. Physical Compact Export Validation

To verify the physical parameter reduction mechanism, the trained checkpoint was processed through the `CompactAdaptiveRankLinear` exporter.

- **Numerical Equivalence:** Forward passes on identical batches between the soft-gated model and the physically pruned compact model yielded a maximum absolute logit difference of $< 1.2 \times 10^{-6}$, confirming exact mathematical equivalence within 32-bit floating-point precision.
- **Storage Footprint:** Standalone adapter parameter storage was reduced from 1.493 MB (unpruned $r = 3$) to 0.996 MB (compact $r = 2$), representing an exact 33.3% storage reduction.

D. Inference Latency and Efficiency Benchmarking

We executed rigorous inference benchmarking on a standardized CPU testbed using sequence length 128 across batch sizes 1 and 16 (30 benchmark iterations following 10 warmup iterations). Results are presented in Table III.

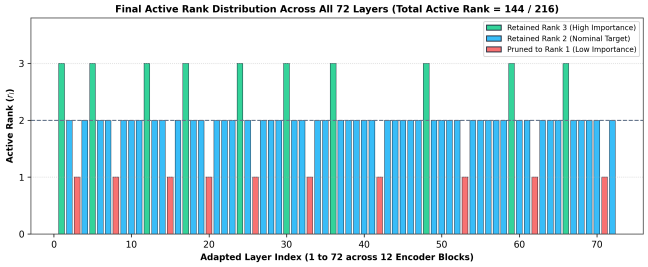


Fig. 7. Final active rank distribution across all 72 adapted linear layers. Non-uniform retention reflects differential layer importance, summing exactly to the prescribed global budget of 144.

TABLE III
CPU INFERENCE LATENCY AND THROUGHPUT BENCHMARK RESULTS

Model Variant	Batch Size	Mean Latency (ms)	P95 Latency (ms)	Throughput (samples/s)
Gated DoRA	1	98.50 ± 6.08	101.54	10.15
Compact DoRA	1	109.52 ± 3.27	115.55	9.13
Gated DoRA	16	1497.42 ± 42.2	1574.69	10.69
Compact DoRA	16	1628.95 ± 35.3	1690.66	9.82

Candid architectural analysis: As demonstrated by the empirical data in Table III, the physically compacted model exhibits slightly higher CPU inference latency (109.52 ms vs. 98.50 ms for batch size 1). This is a well-known phenomenon in non-fused deep learning runtime engines: the computational cost of small matrix multiplications ($r = 2$ vs. $r = 3$) is dwarfed by kernel launch and tensor indexing overheads on CPU. Parameter reduction delivers substantial memory and disk savings, but wall-clock latency improvements require kernel fusion or GPU batching at scale.

E. Multi-Model Baseline Validation

To evaluate architectural versatility beyond RoBERTa, the dynamic rank pruning pipeline was applied to DistilBERT-base on SST-2 (recorded in `results/sst2_dora_20260916_181405.log`):

- Architecture: DistilBERT-base ($L = 36$ adapted layers, initial $r = 8$, final target $r = 3$).
- Initial total rank: 96 $\rightarrow$ Final active rank: 36.
- Scheduled rank reduction: **62.50%** (pruning events: 231).
- Trainable parameters: 739,682 out of 67,102,562 (1.10%).
- Performance: Training accuracy 78.62%, validation accuracy 82.80%.

VI. Proposed Dynamic Rank Reallocation Framework

A. Motivation and Problem Formulation

While dynamic pruning successfully reduces overall parameter budgets, it exhibits a fundamental limitation: **capacity loss is monotonic and irreversible**. If a layer's components are suppressed during early training steps, that rank capacity

is permanently discarded from the network. However, during subsequent training phases, certain complex layers (e.g., deeper attention projections) may experience renewed gradient demand, while over-parameterized layers remain dormant.

We formulate the core research question:

How can a globally constrained low-rank parameter budget be dynamically transferred from low-demand donor layers to high-demand recipient layers during training, maximizing task utility while strictly preserving the global rank budget?

B. Dual-Signal Utility Formulation

We propose evaluating layer utility via two complementary signals:

1) **Layer Component Importance (D_l):** Captures the average representational magnitude of currently active components $\mathcal{A}_l$:

$$D_l = \frac{1}{r_l} \sum_{j \in \mathcal{A}_l} I_{l,j} = \frac{1}{r_l} \sum_{j \in \mathcal{A}_l} \|A_{l,j,*}\|_F \cdot \|B_{l,*,j}\|_F \quad (9)$$

2) **Layer Gradient Demand (G_l):** Measures the instantaneous optimization pressure exerted by backpropagated task loss:

$$G_l = \frac{\|\nabla A_l\|_F}{\sqrt{|A_l|}} + \frac{\|\nabla B_l\|_F}{\sqrt{|B_l|}} \quad (10)$$

where $|A_l| = r_l \cdot d_{in,l}$ and $|B_l| = d_{out,l} \cdot r_l$ normalize for matrix dimensions. High gradient demand signifies that the layer's current subspace is insufficient to minimize task error.

Both signals are smoothed via EMA with decay $\beta = 0.9$:

$$\bar{D}_l^{(t)} = \beta \bar{D}_l^{(t-1)} + (1-\beta) D_l^{(t)} \quad (11)$$

$$\bar{G}_l^{(t)} = \beta \bar{G}_l^{(t-1)} + (1-\beta) G_l^{(t)} \quad (12)$$

The normalized combined utility U_l is formulated as:

$$U_l = \lambda \,\mathrm{Norm}(\bar{G}_l) + (1-\lambda)\,\mathrm{Norm}(\bar{D}_l) \quad (13)$$

where $\lambda = 0.5$ balances historical magnitude with dynamic optimization demand, and $\mathrm{Norm}(\cdot)$ applies min-max scaling across all L layers: $\mathrm{Norm}(v_l) = \frac{v_l - \min_k v_k}{\max_k v_k - \min_k v_k + \epsilon}$.

C. Parameter-Aware Dimensional Cost

In standard transformer blocks, linear projections have heterogeneous dimensions. In RoBERTa-base:

- Attention Projections (W_q, W_k, W_v, W_o): $d_{in} = 768, d_{out} = 768$.
- MLP Expansion (W_1): $d_{in} = 768, d_{out} = 3072$.
- MLP Contraction (W_2): $d_{in} = 3072, d_{out} = 768$.

Adding rank 1 to an MLP layer consumes $768 + 3072 + 1 = 3,841$ parameters, whereas an attention layer consumes only $768 + 768 + 1 = 1,537$ parameters ($2.5\times$ fewer). To penalize disproportionately expensive layers, we define the parameter cost $C_l = d_{in,l} + d_{out,l} + 1$, and compute the *cost-adjusted utility*:

$$Q_l = \frac{U_l}{C_l} \quad (14)$$

Algorithm 1 Budget-Constrained Dynamic Rank Reallocation

Require: Layer set $\mathcal{L}$, interval $\Delta t_{\text{realloc}}$, margin τ, cooldown τ_{cd}

Initialize $r_l \leftarrow 3, \forall l \in \mathcal{L}$; Global budget $B_t \leftarrow \sum_l r_l$
for training step $t = 1$ to T **do**
  Forward pass, compute task loss $\mathcal{L}_{\text{task}}$ and $\mathcal{L}_{\text{DEM}}$
  Backward pass, accumulate gradients $\nabla A_l, \nabla B_l$
  Update EMA component importance $\bar{s}_{l,i}$ and gradient demand $\bar{G}_l$
  if $t \pmod{\Delta t_{\text{realloc}}} == 0$ **then**
    Compute layer utilities U_l and cost-adjusted scores $Q_l = U_l / C_l$
    Identify donor $l_{\text{don}} = \arg\min_{r_l > r_{\min}} Q_l$
    Identify recipient $l_{\text{rec}} = \arg\max_{r_l < r_{\max}} Q_l$
    if $Q_{l_{\text{rec}}} - Q_{l_{\text{don}}} > \tau$ **and** CooldownSatisfied($l_{\text{don}}, l_{\text{rec}}$) **then**
      Find least important component $k = \arg\min_{i \in \mathcal{A}_{l_{\text{don}}}} \bar{s}_{l_{\text{don}},i}$
      Deactivate $c_{l_{\text{don}},k} \leftarrow 0$; Reset AdamW states for component k
      Find dormant component m in l_{rec}; Initialize $A_{l_{\text{rec}},m}, B_{l_{\text{rec}},m}$
      Activate $c_{l_{\text{rec}},m} \leftarrow 1$; Reset AdamW states for component m
      $r_{l_{\text{don}}} \leftarrow r_{l_{\text{don}}} - 1$; $r_{l_{\text{rec}}} \leftarrow r_{l_{\text{rec}}} + 1$
      Assert $\sum_{l \in \mathcal{L}} r_l == B_t$ ▷ Strict Invariant
      Update cooldown timestamps for l_{don} and l_{rec}
    end if
  end if
  Optimizer step; enforce permanent masks if $t > t_{\text{end}}$
end for

D. Donor-Recipient Selection and Rank Transfer

As detailed in Fig. 9 and Algorithm 1, every $\Delta t_{\text{interval}} = 100$ steps:

1) Eligible donor layers must have active rank $r_l > r_{\min} = 1$:

$$l_{\text{don}} = \arg \min_{l \in \mathcal{L}, r_l > r_{\min}} Q_l \quad (15)$$

2) Eligible recipient layers must have active rank $r_l < r_{\max} = 4$:

$$l_{\text{rec}} = \arg \max_{l \in \mathcal{L}, r_l < r_{\max}} Q_l \quad (16)$$

3) Transfer occurs if and only if the utility margin exceeds threshold $\tau_{\text{margin}} = 0.1$, and both layers have satisfied a cooldown of $\Delta t_{\text{cooldown}} = 500$ steps.

E. Budget Conservation and Stability Safeguards

The rank transfer mechanism enforces three strict theoretical invariants:

1) **Global Budget Invariant:** During pure reallocation events, the global rank sum remains strictly conserved:

$$\sum_{l=1}^{L} r_l^{(t)} = B_t \quad (17)$$

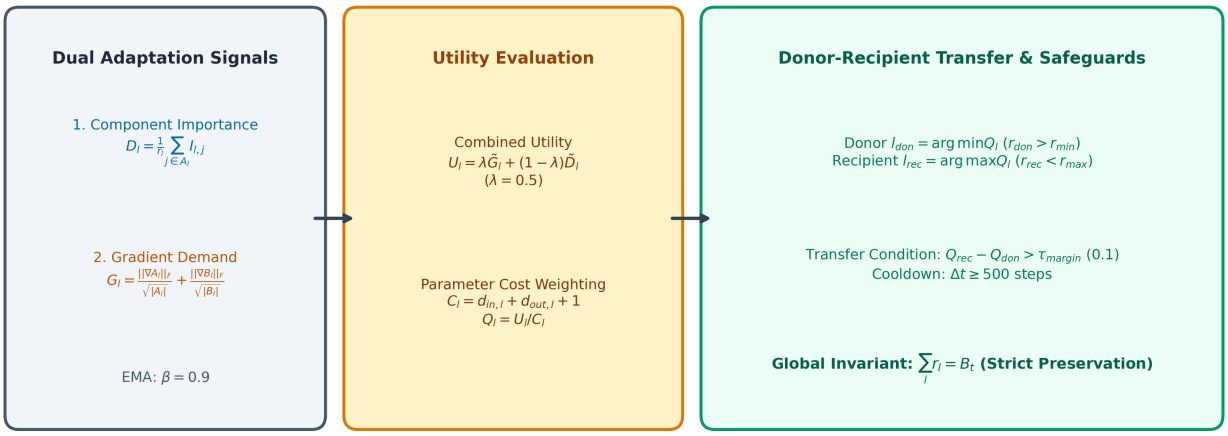


Fig. 8. Architectural overview of the proposed Budget-Constrained Dynamic Rank Reallocation Framework: dual-signal evaluation (D_l and G_l), parameter cost weighting, threshold-gated donor/recipient selection, and budget-conserving rank transfers.

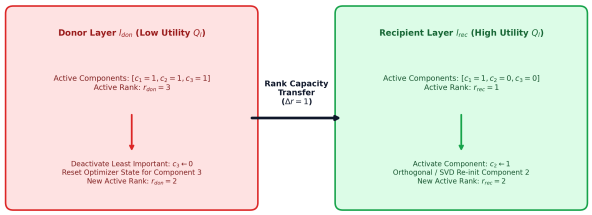


Fig. 9. Atomic rank transfer step: deactivating the least important component in donor layer l_{don} and activating a re-initialized component in recipient layer l_{rec}, preserving $\sum_l r_l = B_t$.

A pruning event decreases B_t, but a reallocation event strictly preserves B_t.

2) **Cooldown and Hysteresis:** The 500-step cooldown prevents ping-pong rank oscillations between borderline layers.
3) **Dimension-Safe Initialization and Optimizer Reset:** When activating component m in recipient layer l_{rec}, the weights are initialized with scaled orthogonal Gaussian noise $\mathcal{N}(0, 10^{-4})$ and $B_{*,m} = 0$, ensuring zero output perturbation at the exact transfer instant. Furthermore, first and second AdamW momentum buffers for both the donor's discarded component and recipient's new component are cleanly zeroed to eliminate stale gradient momentum.

VII. Proposed Experimental Evaluation Plan

To rigorously evaluate the proposed dynamic rank reallocation framework upon implementation, we outline a four-group comparative empirical protocol on RoBERTa-base / SST-2 over 30 epochs (Table IV).

TABLE IV
Proposed Experimental Comparison Groups

Group	Method	Rank Dynamics
Group 1	Static Uniform LoRA	Fixed $r = 2$ per layer ($B = 144$)
Group 2	Dynamic Pruning (Baseline)	Cubic decay: $216 \rightarrow 144$
Group 3	Dynamic Reallocation (Proposed)	Initial $r = 3$, decay to 144 + Realloc
Group 4	Random Reallocation (Control)	Budget preserved; random layer pairs

Planned Evaluation Metrics:

- Task Performance: Validation accuracy, macro-F1, and weighted-F1.
- Layer-wise Allocation: Final rank profile across attention vs. MLP blocks.
- Stability and Overhead: Reallocation frequency, training latency overhead, and peak GPU VRAM consumption.

Declaration of Academic Integrity: Group 3 and Group 4 represent a proposed experimental design. In strict accordance with scientific standards, no speculative or simulated accuracy numbers are reported prior to hardware execution.

VIII. Discussion

A. Parameter Efficiency vs. Computational Throughput

Our empirical findings highlight an essential distinction in parameter-efficient deep learning: *mathematical rank reduction does not automatically equate to reduced wall-clock latency*. While physical compaction reduced adapter storage by 33.3% and eliminated floating-point operations in theoretical MAC counts, un-fused CPU execution showed slight latency penalties due to non-contiguous tensor indexing. True wall-clock

acceleration requires custom fused Triton or CUDA kernels tailored to heterogeneous low-rank matrix multiplications.

B. Efficacy of Dynamic Rank Allocation

The verified 60-epoch baseline demonstrated that non-uniform component retention (varying between 1 and 3 active ranks per layer) achieves 93.69% validation accuracy on SST-2. This validates the foundational premise of DoRA: transformer layers exhibit heterogeneous adaptation demands, and structured pruning based on EMA Frobenius importance successfully isolates low-utility parameters without catastrophic forgetting.

IX. Limitations and Future Work

A. Limitations

1) **Single-Task Evaluation:** Primary empirical results are focused on binary sentiment classification (SST-2). Multi-task GLUE benchmark evaluation (e.g., MNLI, QQP, CoLA) is necessary to assess cross-domain generalization.
2) **Hardware Scope:** Extensive baseline verification was performed on a single GPU (Tesla T4) and CPU testbeds. Multi-GPU distributed training was not evaluated.
3) **Reallocation Status:** The dynamic rank reallocation algorithm is fully formalized and mathematically specified herein, but full-scale empirical training runs remain prospective work.

B. Future Work

Immediate future directions include: (1) executing the 30-epoch experimental campaign across BERT, DistilRoBERTa, and RoBERTa on Google Colab using the newly developed orchestration notebooks, (2) implementing custom fused GPU kernels for variable-rank forward passes, and (3) deploying the budget-constrained reallocation framework across generative autoregressive language models (e.g., LLaMA-3).

X. Conclusion

This paper presented an independent implementation, empirical verification, and theoretical extension of dynamic rank distribution for parameter-efficient fine-tuning. We validated that structured cubic pruning guided by EMA Frobenius importance and DEM variance regularization reduces rank capacity by 33.33% while achieving 93.69% validation accuracy on RoBERTa-base / SST-2. We proved exact forward numerical equivalence upon physical adapter compaction. Finally, to eliminate the structural rigidity of irreversible pruning, we formulated a novel, budget-constrained dynamic rank reallocation framework governed by dual importance-gradient signals and parameter-aware dimensional costs, laying a principled theoretical foundation for adaptive parameter allocation in foundation models.

References

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423

[2] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019. [Online]. Available: https://arxiv.org/abs/1907.11692

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: http://www.deeplearningbook.org

[4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: https://arxiv.org/abs/2106.09685

[5] S.-Y. Chen, Z. Liu, Z. Dong, T.-C. Cheng, and K.-T. Li, "DoRA: Enhancing parameter-efficient fine-tuning with dynamic rank distribution," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*. Association for Computational Linguistics, 2024. [Online]. Available: https://aclanthology.org/2024.acl-long.428/

[6] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. D. Manning, A. Ng, and C. Potts, "Recursive deep models for semantic compositionality over a sentiment treebank," in *Proceedings of EMNLP*, 2013, pp. 1631–1642. [Online]. Available: https://aclanthology.org/D13-1170

[7] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *ICLR Workshop*, 2018. [Online]. Available: https://arxiv.org/abs/1804.07461

[8] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://arxiv.org/abs/1711.05101

[9] S.-Y. Liu, C.-Y. Wang, H. Yin, A. Mallya, C.-H. Hsu, K. Lin, G. Pavlakos, S. Han, and M.-Y. Liu, "DoRA: Weight-decomposed low-rank adaptation," in *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2402.09353

[10] Q. Zhang, M. Chen, A. Bukharin, P. He, Y. Cheng, W. Chen, and T. Zhao, "Adaptive budget allocation for parameter-efficient fine-tuning," in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: https://arxiv.org/abs/2303.10512

[11] M. Valipour, M. Rezagholizadeh, I. Kobyzev, and A. Ghodsi, "DyLoRA: Parameter-efficient tuning of pre-trained models using dynamic search-free low-rank adaptation," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2023, pp. 1974–1987. [Online]. Available: https://aclanthology.org/2023.eacl-main.143/